# 다듬(DaDeum) 완전정복

0장. 이 책을 읽는 법과 전체 지도
- 0.1 이 책의 약속
- 0.2 다듬은 무엇인가
- 0.3 전체 지도
- 0.4 이 책의 구성

1장. 코드 기초: 이 프로젝트를 읽는 데 필요한 것만
- 1.1 프로그램은 무엇인가
- 1.2 변수: 값에 이름 붙이기
- 1.3 자료형: 문자열, 숫자, 참/거짓
- 1.4 조건문: 만약 ~라면
- 1.5 반복문: 여러 번 하기
- 1.6 목록·집합·사전
- 1.7 함수: 이름 붙인 작업 묶음
- 1.8 클래스와 dataclass: 묶음 자료
- 1.9 모듈과 import
- 1.10 정규식: 글자 모양 찾기
- 1.11 정렬과 람다
- 1.12 예외 처리
- 1.13 이 장에서 못 다룬 것 (나오면 그때 설명합니다)
- 1.14 확인 문제

# 0장. 이 책을 읽는 법과 전체 지도

## 0.1 이 책의 약속

이 책은 **코드를 한 줄도 모르는 사람**이 끝까지 읽으면 다듬(DaDeum)의 코드를 전부 설명할 수 있게 되는 것을 목표로 합니다. 그래서 모든 장이 같은 순서로 진행됩니다.

| 순서 | 묻는 것 | 예 |
|---|---|---|
| 1 | **무엇을** 하는가 (목적) | "받침을 보고 을/를을 고른다" |
| 2 | **왜** 필요한가 (이유) | "이 규칙이 없으면 `학교을` 을 못 잡는다" |
| 3 | **어떻게** 하는가 (로직) | 실제 코드와 한 줄씩 풀이 |
| 4 | **어떻게 확인했는가** (근거) | 실제로 돌려 본 결과, 측정 수치 |
| 5 | **무엇이 문제였고 어떻게 풀었는가** (해결) | 실패 사례와 고친 방법 |

코드 인용은 모두 실제 저장소(`zmflm0110/dadeum`, master 6af0ea5)에서 가져왔습니다. 설명에 나오는 실행 결과는 제가 그 코드를 직접 돌려서 얻은 값입니다. 돌려 보지 않은 것은 "읽은 바로는"이라고 적습니다.

> 이 책에서 가장 중요한 습관 하나: **"그렇다더라"로 넘어가지 않고, 입력을 넣고 출력을 보는 것.** 다듬 프로젝트 자체가 그 습관으로 만들어졌습니다. (13장에서 그 이야기를 합니다.)

## 0.2 다듬은 무엇인가

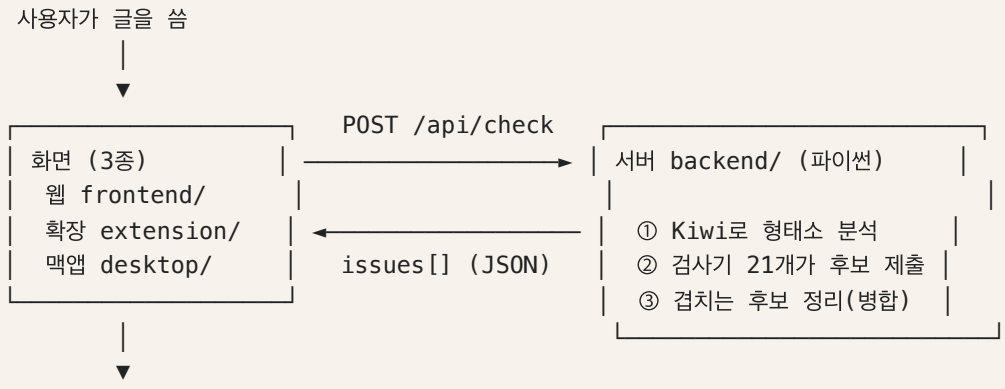
한국어 글을 넣으면 **맞춤법·띄어쓰기·조사·문법 오류에 밑줄을 긋고, 고친 말을 제안하는 프로그램**입니다. 웹 페이지, 크롬 확장, 맥 앱 세 가지 모습이 있고, 셋 다 같은 "엔진"을 씁니다. 전부 내 컴퓨터 안에서 돌고 외부 AI를 부르지 않습니다.

한 문장을 넣으면 이렇게 됩니다.

```
입력:  학교을 가는길에 친구를 만났는데 그렇게 하면 안되요.

출력:  2-3   '을'     → '를'      (조사, 확실)
       25-26 '안'     → '안 '     (띄어쓰기, 확실)
       26-28 '되요'   → '돼요'    (맞춤법, 확실)
       13-17 '만났는데' → (없음)   (이상한 말, 애매)
```

## 0.3 전체 지도




| 폴더 | 한 줄 설명 | 이 책의 장 |
|---|---|---|
| `backend/app/` | 교정 엔진 본체 | 4-10장 |
| `backend/ml/` | 모델을 학습시키는 코드 | 7-9장 |
| `backend/eval/` | 엔진이 좋은지 재는 도구 | 11장 |
| `backend/tests/` | 자동 테스트 997개 | 11장 |
| `core/` | 화면 세 개가 공유하는 상태기계 | 12장 |
| `frontend/` `extension/` `desktop/` | 화면 | 12장 |

## 0.4 이 책의 구성

| 장 | 내용 |
|---|---|
| 1 | 코드 기초: 이 프로젝트를 읽는 데 필요한 파이썬 문법만 |
| 2 | 한글이 컴퓨터 안에서 어떻게 생겼나 (받침 계산의 원리) |
| 3 | 형태소 분석: 문장을 조각내고 이름표를 붙이기 |
| 4 | 서버와 API: 요청 하나가 지나가는 길 |
| 5 | 엔진 뼈대: 후보를 모으고 정리하는 구조 |
| 6 | 규칙 검사기: 조사·자모·사전·의존명사 등 |
| 7 | 머신러닝 기초와 혼동쌍 모델 |
| 8 | 단어 경계 모델 |
| 9 | 오타·사전·언어모델·이상탐지 |
| 10 | 띄어쓰기 |
| 11 | 평가: 좋은지 어떻게 아는가 |
| 12 | 화면과 공유 코어 |
| 13 | 개발 과정과 교훈 |
| 14 | 용어집, 직접 돌려 보기, 연습문제 |

앞 장을 건너뛰면 뒤 장의 용어를 못 알아볼 수 있습니다. 처음엔 순서대로 읽으세요.

# 1장. 코드 기초: 이 프로젝트를 읽는 데 필요한 것만

다듬의 엔진은 **파이썬**으로 쓰여 있습니다. 파이썬 전체를 배울 필요는 없고, 이 프로젝트에 실제로 나오는 문법만 알면 됩니다. 각 항목마다 "다듬 코드의 어디에 나오는지"를 같이 적었습니다.

## 1.1 프로그램은 무엇인가

프로그램은 **입력을 받아 정해진 순서대로 처리해서 출력을 내는 글**입니다. 요리 레시피와 같습니다.

```
입력: "학교을 갔다"  →  [처리: 받침을 보고 을/를 고르기]  →  출력: "학교를 갔다"
```

컴퓨터는 사람처럼 "대충" 이해하지 못합니다. 모든 단계를 정확히 적어 줘야 합니다. 다듬 코드가 길어 보이는 이유는 "이 경우는 이렇게, 저 경우는 저렇게"를 전부 적었기 때문입니다.

## 1.2 변수: 값에 이름 붙이기

```
text = "학교을 갔다"
count = 3
```

`text`라는 이름표를 값에 붙인 것입니다. `=`는 "같다"가 아니라 **"오른쪽 값을 왼쪽 이름에 넣어라"**입니다.

다듬 예: `app/checkers/confusion.py`에는 `_DOE_FORCES_DOE = set("고게기니나냐네는면며자거든어었겠시")`가 있습니다. "되 뒤에 이 글자들이 오면 무조건 '되'"라는 목록에 이름을 붙인 것입니다.

## 1.3 자료형: 문자열, 숫자, 참/거짓

| 종류 | 예 | 다듬에서 |
|---|---|---|
| 문자열 `str` | `"학교"` | 입력 글, 제안, 설명 메시지 |
| 정수 `int` | `26` | 글자 위치(start, end) |
| 실수 `float` | `0.97` | 확률, 문턱 |
| 참/거짓 `bool` | `True` | 조건 검사 결과 |
| 없음 `None` | `None` | "답이 없다" (예: 받침을 알 수 없는 글자) |

**문자열의 번호(인덱스)는 0부터 셉니다.**

```
text = "학교을 갔다"
text[0]      # '학'
text[2]      # '을'
text[0:2]    # '학교'   (0번부터 2번 "직전"까지)
```

다듬이 말하는 `start=2, end=3`은 "2번 글자부터 3번 직전까지", 즉 `text[2:3]` = `'을'` 입니다. 이 "끝은 포함하지 않는다" 규칙이 프로젝트 전체에서 일관되게 쓰입니다.

## 1.4 조건문: 만약 ~라면

```
if idx == NO_BATCHIM_INDEX:
    answer = "를"
else:
    answer = "을"
```

`==`는 "같은가?"를 묻는 기호입니다(`=`와 다릅니다). 다듬 코드 거의 모든 줄이 이런 조건의 연속입니다.

`and`(둘 다), `or`(하나라도), `not`(반대)로 조건을 묶습니다.

```
if actual in (no_batchim, has_batchim) and actual != expected:
    ...   # 조사가 둘 중 하나이면서, 기대한 것과 다르면
```

## 1.5 반복문: 여러 번 하기

```
for ch in "학교을":
    print(ch)
# 학
# 교
# 을
```

`for 이름 in 목록:`은 목록의 원소를 하나씩 꺼내 아래 들여쓴 줄을 실행합니다. **들여쓰기(맨 앞 공백)가 문법입니다.** 파이썬은 괄호 대신 들여쓰기로 "여기까지가 for의 안쪽"을 표시합니다.

`continue`는 "이번 차례는 건너뛰고 다음으로", `break`는 "반복을 완전히 끝내기"입니다. 다음의 검사기들은 `if (오탐 위험 조건): continue`로 위험한 자리를 건너뛰는 모양이 아주 많습니다.

## 1.6 목록·집합·사전

```
issues = [ ... ]            # 목록(list): 순서 있음, 같은 값 여러 개 가능
seen = {3, 7, 9}            # 집합(set): 순서 없음, 중복 없음, "있나?" 확인이 빠름
rule = {"pattern": "결재", "replacement": "결제"}   # 사전(dict): 이름표 → 값
```

| | 만들기 | 꺼내기 | 다듬 예 |
|---|---|---|---|
| list | `[1, 2]` | `a[0]` | 검사기가 낸 `issues` |
| set | `{1, 2}` | `3 in a` | `seen_spans` : 이미 본 위치 |
| dict | `{"k": v}` | `d["k"]` | `dict_rules.json`의 규칙 한 개 |

**리스트 컴프리헨션**은 "목록을 걸러서 새 목록 만들기"를 한 줄로 쓴 것입니다.

```
actionable = [i for i in issues if i.confidence != "low"]
# 풀어 쓰면:
actionable = []
for i in issues:
    if i.confidence != "low":
        actionable.append(i)
```

## 1.7 함수: 이름 붙인 작업 묶음

```python
def final_index(ch):
    """받침 번호를 돌려준다."""
    if not is_hangul_syllable(ch):
        return None
    return (ord(ch) - 0xAC00) % 28
```

- `def 이름(입력):`으로 만듭니다. `return`은 결과를 돌려주고 함수를 끝냅니다.
- 따옴표 세 개 `"""..."""`는 **설명문(독스트링)**입니다. 실행에 영향이 없고 읽는 사람을 위한 것입니다. 다듬 코드는 이 설명이 매우 깁니다. 왜 이렇게 했는지, 어떤 실패가 있었는지가 거기 적혀 있습니다.
- `#` 뒤는 한 줄 주석입니다.

함수를 쓰는 것을 **호출**이라고 합니다: `final_index("교")` → `0`.

### 타입 힌트

```python
def check_josa(text: str, kiwi: Kiwi) -> list[RawIssue]:
```

`: str`은 "이 입력은 문자열", `-> list[RawIssue]`는 "결과는 RawIssue의 목록"이라는 **표시**입니다. 실행에 영향은 없고 읽기 쉽게 하려는 것입니다. `str | None`은 "문자열이거나 None"입니다.

## 1.8 클래스와 dataclass: 묶음 자료

관련된 값 여러 개를 하나로 묶고 싶을 때 씁니다. 다듬의 핵심 자료인 `RawIssue`가 이렇게 생겼습니다(`app/checkers/base.py`).

```python
from dataclasses import dataclass

@dataclass
class RawIssue:
    start: int
    end: int
    category: str
    source: str
    original: str
    suggestions: list[str]
    message: str
    confidence: str
```

"지적 한 건"은 위치(start, end), 종류(category), 출처(source), 원문(original), 고칠 말(suggestions), 설명(message), 확신도(confidence)로 이루어집니다.

```
r = RawIssue(start=2, end=3, category="josa", source="josa_rule",
             original="을", suggestions=["를"], message="...", confidence="h
         igh")
r.original      # '을'  ← 점(.)으로 안의 값을 꺼냄
r.suggestions[0]  # '를'
```

`@dataclass`는 "이 클래스의 생성자와 출력 모양은 자동으로 만들어 줘"라는 표시입니다. `@dataclass(frozen=True)`는 "한 번 만들면 값을 못 바꾼다"입니다(`Generator`, `Source`가 이렇습니다).

## 1.9 모듈과 import

파일 하나가 모듈입니다. 다른 파일의 것을 가져다 쓰려면 `import`합니다.

```
from .base import RawIssue          # 같은 폴더의 base.py에서 RawIssue를
from ..hangul import final_index    # 한 단계 위 폴더의 hangul.py에서
from kiwipiepy import Kiwi          # 설치된 외부 라이브러리에서
```

점(`.`)의 개수는 폴더 거리입니다. 한 개는 같은 폴더, 두 개는 위 폴더입니다.

## 1.10 정규식: 글자 모양 찾기

**정규식(regular expression)**은 "이런 모양의 글자를 찾아라"를 적는 작은 언어입니다. 다듬의 사전 규칙 474개가 전부 정규식입니다.

| 기호 | 뜻 | 예 |
|---|---|---|
| `abc` | 그 글자 그대로 | `결재` |
| `A\|B` | A 또는 B | `결재\|결제` |
| `[가-힣]` | 한글 한 글자 | |
| `.` | 아무 글자 | |
| `+` | 앞의 것 1번 이상 | `[가-힣]+` |
| `^` `$` | 문자열의 처음, 끝 | |
| `(?<=X)` | "바로 앞에 X가 있을 때만" (앞을 확인만 하고 소비하지 않음) | |
| `(?=X)` | "바로 뒤에 X가 있을 때만" | |
| `(?!X)` | "바로 뒤에 X가 없을 때만" | |

다듬의 실제 규칙(`data/dict_rules.json`)입니다.

```
(?<=카드로 )결재        → 결제
```

"`카드로` 바로 뒤에 오는 `결재`"만 찾아서 `결제`로 바꿉니다. 앞의 `카드로`는 확인만 하고 바꾸는 범위에는 들어가지 않습니다. 그래서 `서류를 결재`는 건드리지 않습니다. **문맥을 정규식 안에 넣어서** 같은 철자의 다른 쓰임을 구별하는 것입니다.

파이썬에서는 이렇게 씁니다.

```python
import re
pat = re.compile(r"(?<=카드로 )결재")
for m in pat.finditer("카드로 결재했다"):
    print(m.start(), m.end(), m.group(0))   # 4 6 결재
```

## 1.11 정렬과 람다

```python
ordered = sorted(issues, key=lambda i: (i.start, -(i.end - i.start)))
```

`sorted`는 목록을 정렬합니다. `key=`는 "무엇을 기준으로 줄 세울지"입니다. `lambda i: ...`는 이름 없는 한 줄짜리 함수입니다. 위 기준은 `(시작 위치, 길이의 음수)`이므로 **시작이 빠른 것이 먼저, 시작이 같으면 긴 것이 먼저** 옵니다. 5장의 병합이 이 정렬 위에서 돌아갑

니다.

## 1.12 예외 처리

```
try:
    return kiwi.analyze(word)[0][1]
except Exception:
    return 0.0
```

`try` 안에서 오류가 나도 프로그램이 죽지 않고 `except` 안쪽을 실행합니다. 다음은 "분석 한 번이 실패해도 서버 전체가 죽으면 안 된다"는 원칙으로 곳곳에 씁니다.

## 1.13 이 장에서 못 다룬 것 (나오면 그때 설명합니다)

- `yield` (값을 하나씩 내보내는 함수): 8장 경계 후보
- 데코레이터 `@router.post(...)` : 4장 서버
- `async` / `await` (JavaScript): 12장 화면

## 1.14 확인 문제

1. `text = "안녕하세요"` 일 때 `text[1:3]` 은? (답: `녕하` )
2. `start=4, end=6` 인 지적이 가리키는 글자를 `"카드로 결재했다"` 에서 꺼내면? (답: `결재` )
3. `[i for i in [1,2,3,4] if i > 2]` 의 결과는? (답: `[3, 4]` )